MOCA — Template Onboarding Guide

Last modified: 2026-10-31T03:16:43Z

> Hand-maintained. Bump the line above when you edit this guide.

This guide takes a container image to a **sandbox template**: a named image that a client, or an agent session, asks for by id, and whose commands then run in microVMs restored from that image's snapshot. It walks through two worked examples end to end: MOCA's public sandbox image (anyone can reproduce it) and a SWE-bench instance (the benchmark case it was built for). It was validated on 2026-10-29 and 2026-10-30 against an SBX0 deployment: the control plane and relay on OpenShift, and the workers on a bare-metal KVM host.

- **A template is one worker serving one snapshot.** Onboarding is: build a snapshot from the image, check it in a real microVM, give it a worker, register it, use it, and retire it when a new image replaces it.
- **Everything for templates is in this folder:** the tools, the two sample specs, their tests, and links to the related files elsewhere.
- The short reference for the same steps is `deploy/sbx/template-onboarding/ONBOARDING.md`. Deploying SBX0 itself is `deploy/sbx/README.md` and `RUNBOOK.md`; using a sandbox is the user guide; templates in MOCA's own agent sessions are designed in `2026-10-30-session-sandbox-templates-design.md`.
- **Every host and registry below is a placeholder:** `sbx-api.example.com`, `registry.example.com`. No real host, account id or token appears here.

In a hurry? §2 is the whole flow on one page; §4 is the public-image example as copy-pasteable commands; §6 is a client choosing between the two sample templates.

Contents

- 1. Mental model (read this first)
  - Where things run
  - Names
- 2. The flow on one page
- 3. The template spec
  - The probe must be shareable
- 4. Example 1: MOCA's public sandbox image
  - 4.1 Write the spec
  - 4.2 Build and probe on the KVM host
  - 4.3 Register (operator machine)
  - 4.4 Install the worker
  - 4.5 Use it
- 5. Example 2: a SWE-bench instance (django__django-11133)
  - 5.1 The spec comes from image-factory
  - 5.2 Build, register, install
  - 5.3 A client: seed first

    - 5.4 An agent: OpenCode on the template
- 6. Two templates side by side: the client chooses
- 7. Update and retire
- 8. Templates in MOCA agent sessions
- 9. Capacity
- 10. Troubleshooting (what we actually hit)
- 11. Regenerating the documents

---

1. Mental model (read this first)

1. **A sandbox runs every command in a fresh microVM** restored from its template's golden snapshot, with the sandbox's own `/workspace` attached. Files in `/workspace` persist across commands; nothing else does.
2. **One worker (`microvm-worker`) serves one snapshot.** A template is a name for a worker, so a new image of a template is a new snapshot and a new worker; the old one drains.
3. **The client chooses the template** when it creates a sandbox (`mocactl sbx create --template ID`, `POST /v1/sandboxes {template}`), and a sandbox keeps its worker for life.

Where things run

Component	Runs on	Does
`template-build.sh`	the KVM host, as root	image → snapshot, checked in a microVM → a template record
`template-register.sh`	the operator's machine	record → SBX0's inputs (SBX_WORKERS, the templates file)
`setup.sh`	the operator's machine, against the cluster	mints the worker's token and bundle; restarts the control plane only
`install-host.sh`	the KVM host, as root	installs the worker, which attaches to the relay
`mocactl sbx`	anywhere	lists templates, creates sandboxes, runs commands

Names

For a template id `ID` and an image digest `sha256:abcdef012345…`:

What	Name
snapshot	`ID-abcdef012345` (the first 12 hex of the digest)
worker id	`sbx_<ID with - as _>_abcdef012345`
record	`$SBX_SRV/templates/<snapshot>.json`

2. The flow on one page

#	Where	Command	Check
1	anywhere	write `spec.json` (§3)	`template-build.sh` validates every field first
2	KVM host	`template-build.sh spec.json --kernel K [--registry H]`	digest, `/etc/hosts`, the probe in a microVM; a failure publishes nothing
3	operator	`template-register.sh --env-file F record.json`	one worker per template; an update keeps the old worker
4	operator	`setup.sh --env-file F`	the control plane rolls; the relay does not
5	KVM host	`install-host.sh BUNDLE --snapshot S --budget-mb N [--no-egress]`	the worker attaches
6	client	`mocactl sbx create --template ID`	the sandbox runs on that template's image
7	both	new image: steps 1-5 again, then `--retire` the old worker	its live sandboxes have ended

3. The template spec

A JSON file, schema `moca-sbx-template-spec/1`. Every field is checked before anything runs (`template-build.sh`):

Field	Meaning
`id`	what clients ask for: `^[a-z0-9][a-z0-9-]{0,30}$`; the same across versions
`image`	`repository@sha256:<digest>`, never a tag; a repository with no host takes it from `--registry`
`guest_ram_mb`, `vcpus`	the guest's size, fixed in the snapshot
`guest_home`	optional, a writable `HOME` under `/tmp` (the rootfs is read-only)
`skip_git_identity_check`	`true` for an image that refuses agent commits on purpose
`egress`	`false`: no network at all (`--no-egress`); `true`: the host's allowlisted egress
`seed`	a bash command that fills a new sandbox's empty `/workspace`, or `""`
`probe`	`{cmd, timeout_s, expect_rc, expect[]}`: run after the seed in the restored VM; passes on the exit code and every regex
`description`, `tools`	what clients see in `GET /v1/templates`
`etc_hosts_sha256`	optional: the build stops unless the rootfs's `/etc/hosts` has this hash

The probe must be shareable

The probe proves the image's toolchain works in a microVM, not that a task is solvable. For a benchmark, **use a public test target, never the tests that grade a fix**: their names and counts are answer-key material. Match the runner's output as it prints it (§10 has two ways this went wrong).

4. Example 1: MOCA's public sandbox image

MOCA's own sandbox image, `ghcr.io/rossoctl/moca-remote-worker`, is public, so this example needs no registry key. It is the image SBX0's `default` template ran on 2026-10-29 (built then with `build-snapshot.sh` directly); the same flow through `template-build.sh` is *(unverified with this exact spec)*.

4.1 Write the spec

This spec is `examples/default.json` in this folder:

```
{
  "schema": "moca-sbx-template-spec/1",
  "id": "default",
  "image": "ghcr.io/rossoctl/moca-remote-
   worker@sha256:eed901b838bf11e16054fccfb38af600a6e61d74dd52907af0750bba13ecbed3",
  "guest_ram_mb": 256,
  "vcpus": 1,
  "egress": true,
  "seed": "",
  "probe": {
    "cmd": "git --version && python3 -c 'print(6*7)' && rg --version | head -1",
    "timeout_s": 120,
    "expect_rc": 0,
    "expect": ["^git version ", "^42$", "^ripgrep "]
  },
  "description": "MOCA's sandbox image: bash, git, Python 3, ripgrep",
  "tools": ["bash", "git", "python3", "rg"]
}
```

`egress: true` gives the guest the host's allowlisted egress (GitHub by default, RUNBOOK). Use `false` for a template that must stay offline.

4.2 Build and probe on the KVM host

```
sudo deploy/sbx/template-onboarding/template-build.sh deploy/sbx/template-
   onboarding/examples/default.json --kernel /path/to/vmlinux
```

Expected output, ending:

```
build-snapshot.sh: verify command exited 0; the last of its output:
...
template-build: the probe passed
==> built template default: snapshot default-eed901b838bf, worker sbx_default_eed901b838bf
==> record: /srv/sbx/templates/default-eed901b838bf.json (copy it to the operator's machine for
   deploy/sbx/template-onboarding/template-register.sh)
==> then, after setup.sh: sudo deploy/sbx/install-host.sh <bundle of sbx_default_eed901b838bf> -
   -snapshot default-eed901b838bf --budget-mb <at least 1888>
```

A 256 MiB guest's snapshot is small; a 4 GiB one is about 7.3 GB (§9).

4.3 Register (operator machine)

Copy the record over, then:

```
deploy/sbx/template-onboarding/template-register.sh --env-file <env file> default-
   eed901b838bf.json --apply
```

It adds the worker to `SBX_WORKERS` and the template to the templates file, then runs `setup.sh`. Only the control plane restarts, so running commands on other templates are not interrupted (on 2026-10-30 the relay's settings hash stayed `1cf8f5032ae16280` across two template changes).

4.4 Install the worker

Copy the bundle `deploy/sbx/.generated/workers/sbx_default_eed901b838bf` to the host by path (it holds a token: never print it), then:

```
sudo deploy/sbx/install-host.sh <bundle dir> --snapshot default-eed901b838bf --budget-mb 8192
```

Expected: `sbx-worker@sbx_default_eed901b838bf attached to the relay`.

4.5 Use it

```
mocactl sbx templates
mocactl sbx create --template default --name try-default
mocactl sbx run -- python3 -c 'print(6*7)'
```

5. Example 2: a SWE-bench instance (django__django-11133)

This is exactly what ran on 2026-10-30 (the record is in the project's run notes): a SWE-bench Lite instance, built, checked in a microVM, registered, used by a client and by an agent, then retired.

5.1 The spec comes from image-factory

The result, as it ran, is `examples/django__django-11133.json` in this folder (its image names no registry host; give it with `--registry`).

image-factory's moca-microvm records carry the spec under `template`, so nothing is written by hand. Its `probe` is a **public** test target (`basic`), not the instance's grading tests, and its image names no registry host:

```
jq '.template' platforms/moca/microvm/django__django-11133.json > django__django-11133.json
```

The fields that matter: `id: django-django-11133` (the instance id lower-cased, `__` → `-`; a Verified image also in Lite gets `-v`), 4096 MiB and 2 vCPUs, `egress: false`, `seed: "cp -a /opt/testbed-src/. /workspace/"`, `probe.expect: ["^OK\\b"]` with `expect_rc: 0`, and `etc_hosts_sha256`.

5.2 Build, register, install

```
sudo deploy/sbx/template-onboarding/template-build.sh deploy/sbx/template-
   onboarding/examples/django__django-11133.json --kernel /path/to/vmlinux --registry
   registry.example.com
```

As it ran on 2026-10-30 (about 3 minutes, plus the pull):

```
==> etc/hosts as the spec says
```

```
build-snapshot.sh: verify command exited 0; the last of its output:
Ran 53 tests in 0.062s
OK (skipped=2)
template-build: the probe passed
==> built template django-django-11133: snapshot django-django-11133-3d3e705ff379, worker
   sbx_django_django_11133_3d3e705ff379
```

Then §4.3 and §4.4 with that record, and **`--no-egress` with `--budget-mb 13408`**: a 4 GiB guest needs (4096 + 32) × 3 + 1024 MiB for one session at standby depth 2.

5.3 A client: seed first

A new sandbox's `/workspace` is empty. **SBX0 does not run the seed: the client does**, as its first command. And use `sbx create`, not `sbx up`: `up` pushes the client's own repository into `/workspace` before the seed runs.

```
id=$(mocactl sbx create --template django-django-11133 --json | jq -r .id)
mocactl sbx run --sandbox "$id" -- bash -c 'cp -a /opt/testbed-src/. /workspace/ && git -C
   /workspace log --oneline -1'
mocactl sbx run --sandbox "$id" -- bash -c 'source /opt/miniconda3/bin/activate && conda
   activate testbed && cd /testbed && ./tests/runtests.py --verbosity 2 --settings=test_sqlite -
   -parallel 1 basic 2>&1 | tail -3'
```

Expected (2026-10-30): the seed in 1-2 s with a clean tree at `bc34802 baseline`, then `Ran 53 tests`, `OK (skipped=2)`. The guest has no network (`connect: Network is unreachable`), and a token for this sandbox is `PERMISSION_DENIED` at any other template's worker.

5.4 An agent: OpenCode on the template

Point the agent's MCP server at the seeded sandbox (`mocactl sbx mcp --sandbox <id>`), keeping `sbx up`'s tools-off block, so the agent works only through the sandbox. On 2026-10-30 a headless `opencode run` with the instance's problem statement used `sandbox_bash`, `sandbox_read` and `sandbox_edit` only, finished in 23 s, and its patch was **graded resolved** by the official SWE-bench harness. Take the patch from the sandbox (`git diff --binary <baseline>` in `/workspace`), not with `sbx pull`, which diffs against the client's repository.

6. Two templates side by side: the client chooses

With both examples onboarded, the deployment serves two templates, each on its own worker. The templates file that `template-register.sh` wrote holds:

```
[
  {
    "id": "default",
    "worker": "sbx_default_eed901b838bf",
    "description": "MOCA's sandbox image: bash, git, Python 3, ripgrep",
    "tools": ["bash", "git", "python3", "rg"]
  },
  {
    "id": "django-django-11133",
    "worker": "sbx_django_django_11133_3d3e705ff379",
    "description": "SWE-bench django__django-11133 (lite) at its baseline bc348023edb4; seed
   with: cp -a /opt/testbed-src/. /workspace/",
```

```
    "tools": ["bash", "git", "python3", "rg"]
  }
]
```

A client lists them, then picks one per sandbox. The worker ids never reach the client: it sees only the id, the description and the tools.

```
mocactl sbx templates
```

```
default          MOCA's sandbox image: bash, git, Python 3, ripgrep
django-django-11133 SWE-bench django__django-11133 (lite) at its baseline bc348023edb4; seed
   with: cp -a /opt/testbed-src/. /workspace/
```

(`--json` gives the same list as JSON, with `tools`.)

A sandbox of each, and a command that tells them apart (only the SWE-bench image has the instance's repository at `/opt/testbed-src`):

```
gen=$(mocactl sbx create --template default --name general --json | jq -r .id)
swe=$(mocactl sbx create --template django-django-11133 --name task --json | jq -r .id)
for s in "$gen" "$swe"; do
  mocactl sbx run --sandbox "$s" -- bash -c 'git -C /opt/testbed-src log --oneline -1
   2>/dev/null || echo no-testbed'
done
```

Expected:

```
no-testbed
bc34802 baseline
```

The SWE-bench template also has no network (`egress: false`, checked on 2026-10-30: `connect: Network is unreachable`). `default`'s egress (`egress: true`, through the host's allowlisting proxy, so `git` and `curl` over HTTPS rather than raw TCP) is *(unverified)*: the 2026-10-29 `default` worker was installed with `--no-egress`.

What SBX0 guarantees, as checked live with two templates on 2026-10-29 (`deploy/sbx/templates-check.sh`, 7 checks, 0 failed):

- the list names every template and **no worker id**;
- each create records the template asked for; an **unknown template is refused** (`no such template`) and creates nothing;
- each template's sandboxes go to **its own worker**; two sandboxes of one template share the worker but **not their workspaces**;
- each sandbox's commands run on **its template's image** (the probe saw each image's digest);
- a token for one template's sandbox sent to the other template's worker is **`PERMISSION_DENIED`** at the relay;
- another user cannot see or use the sandbox;
- `sbx up --template` refuses a repository already bound to a sandbox of another template.

The 2026-10-29 run used `default` and an earlier SWE-bench template, `swe-astropy-14995`; the 2026-10-30 run repeated the choice and isolation checks with `django-django-11133` and `astropy-astropy-14995`.

To check a deployment yourself, run `templates-check.sh` against two of its templates (RUNBOOK §14).

7. Update and retire

A new image for a template: a new spec with the new digest, then build, register and install again. The record names a new worker; registering moves the template there and **keeps the old worker** so its live sandboxes keep working. Once none is left:

```
deploy/sbx/template-onboarding/template-register.sh --env-file <env file> --retire <old worker
   id> --apply
sudo deploy/sbx/install-host.sh --uninstall <old worker id>
```

Removing a template entirely is `--remove ID`. On 2026-10-30 retiring `django-django-11133` revoked its worker's token, a create of it was refused (`no such template`), and an astropy sandbox created before kept running.

8. Templates in MOCA agent sessions

MOCA's own agent sessions (Moca Pi, P6) can name a template too: `mocactl run --option sandboxTemplate=ID`. A worker serves a template when `SANDBOX_TEMPLATE` is set; a session with no template never lands on one; the harness runs the seed once, before the agent's first tool call. A templated P4 worker is installed beside the main one with `setup-microvm.sh --template ID --snapshot DIR`; set-up is in `P4-ON-P6.md`. This is tested by unit and relay-integration tests; **a live MOCA-VM check is still to come** *(unverified on a live stack)*.

9. Capacity

Limit	Rule	On a 188 GiB / ~100 GB-free host
disk	~7.3 GB per 4 GiB-guest snapshot; `template-build.sh` wants `SBX_MIN_FREE_GB` (50) free	about 13 templates
memory	`--budget-mb` ≥ (guest + 32) × 3 + 1024 per worker; standbys charged at dirty-page cost	to be measured
quotas	`SBX_MAX_PER_SUBJECT`, `SBX_MAX_TOTAL`	deployment settings
logins	a minted login lasts at most 24 h	—

A measured capacity test is planned; until then treat these as estimates.

10. Troubleshooting (what we actually hit)

Symptom	Cause	Fix
`no match for ^Ran 51 tests?\b`, though the tests passed	an expected count rebuilt from "passed", but Ran counts skips	expect what the runner prints (`^OK\b`, or the Ran total)
`no match for \b8 passed\b`, though pytest passed	pytest coloured its summary (`ESC[32m passed`)	fixed in `template-build.sh`: it matches with ANSI stripped

Symptom	Cause	Fix
a probe expectation from the gate's counts	the gate counts include grading tests	use a public target (§3)
`git checkout` fails part way as a non-root user	root-owned files from earlier root builds in the checkout	`chown -R` the checkout to its user first; check with `find … -not -user …`
a worker restarts and fails	its env names a snapshot no longer on disk	point it at one that exists before starting
`sbx up` sandbox has the client's repo, not the template's	`up` pushes the client repository	`sbx create`, then the seed (§5.3)

11. Regenerating the documents

Artifact	Generated by	Source of truth
this guide's table of contents	`python3 docs/tools/gen_toc.py deploy/sbx/template-onboarding/GUIDE.md`	the headings
`deploy/sbx/template-onboarding/GUIDE.pdf`	`python3 docs/tools/gen_pdf.py`	this `.md`

If no generator rebuilds it, don't commit it.